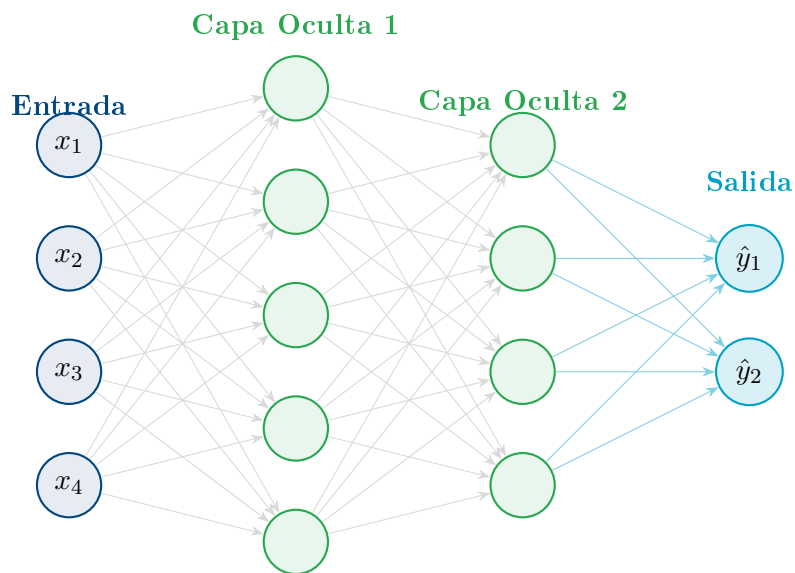

Redes Neuronales Artificiales

De un Perceptrón a una Red Multicapa



Sesión: Redes Neuronales Multicapa (MLP)

Nivel: Intermedio

Modalidad: Apuntes + Práctica en Python

Grupo: Semillero de Machine Learning — UTB

Fecha: April 16, 2026

Contents

1 Motivación: ¿Por qué no basta un perceptrón?

Ya conocemos el perceptrón simple: una neurona que toma entradas $x_1, \dots, x_n$, las pondera con pesos w_i , suma un sesgo b , y pasa el resultado por una función de activación f :

$$\hat{y} = f\left(\sum_{i=1}^n w_i x_i + b\right) = f(\mathbf{w}^\top \mathbf{x} + b).$$

El perceptrón aprende hiperplanos: su frontera de decisión es *lineal*. Esto es suficiente para problemas linealmente separables (AND, OR), pero muchos problemas del mundo real **no** lo son.

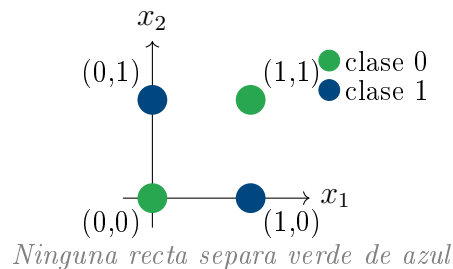
1.1 El problema XOR: el límite del perceptrón

Ejemplo

La función XOR mapea:

$$(0, 0) \rightarrow 0, \quad (0, 1) \rightarrow 1, \quad (1, 0) \rightarrow 1, \quad (1, 1) \rightarrow 0.$$

Intenta encontrar una recta $w_1 x_1 + w_2 x_2 + b = 0$ que separe los 1's de los 0's. ¡Es imposible!



Intuición

Un perceptrón solo puede cortar el espacio con un hiperplano. Para XOR necesitamos **dos cortes**: la solución existe en un espacio transformado. Eso es exactamente lo que hacen las capas ocultas.

2 Redes Neuronales Multicapa (MLP)

2.1 Arquitectura

Una **red neuronal multicapa** (MLP, *Multi-Layer Perceptron*) es una composición de capas:

1. **Capa de entrada**: recibe los datos $\mathbf{x} \in \mathbb{R}^{n_0}$.

2. Capas ocultas: transforman la representación. Una red con $L - 1$ capas ocultas tiene profundidad L .

3. Capa de salida: produce la predicción $\hat{\mathbf{y}}$.

Cada capa ℓ realiza la transformación:

$$\mathbf{z}^{[\ell]} = W^{[\ell]} \mathbf{a}^{[\ell-1]} + \mathbf{b}^{[\ell]}, \quad \mathbf{a}^{[\ell]} = f^{[\ell]}(\mathbf{z}^{[\ell]})$$

donde $W^{[\ell]} \in \mathbb{R}^{n_\ell \times n_{\ell-1}}$, $\mathbf{b}^{[\ell]} \in \mathbb{R}^{n_\ell}$, $\mathbf{a}^{[0]} = \mathbf{x}$.

2.2 Dimensiones de los pesos

Nota

Para una red con capas de tamaños $[n_0, n_1, n_2, \dots, n_L]$:

- $W^{[\ell]}$ tiene dimensión $n_\ell \times n_{\ell-1}$ (filas = neuronas de la capa actual, columnas = capa anterior).
- $\mathbf{b}^{[\ell]}$ tiene dimensión $n_\ell \times 1$.
- El total de parámetros es $\sum_{\ell=1}^L (n_\ell \cdot n_{\ell-1} + n_\ell)$.

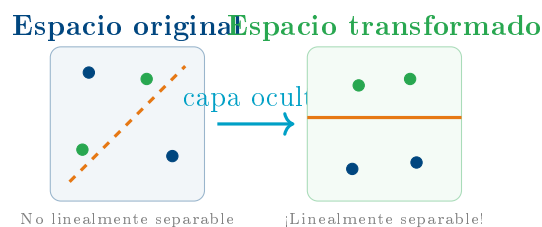
2.3 Intuición geométrica de las capas

Intuición

Cada capa **transforma el espacio** en el que viven los datos:

- La multiplicación por W rota y escala el espacio.
- El sesgo $\mathbf{b}$ lo traslada.
- La función de activación **dobra** o **arruga** el espacio.

Después de suficientes capas, datos que antes no eran linealmente separables, *sí lo son*. Por eso las capas ocultas se llaman a veces *representaciones aprendidas*.



3 Funciones de Activación

3.1 ¿Por qué no usar solo funciones lineales?

Sin activaciones no lineales, una red de L capas seguiría siendo equivalente a **una sola capa lineal**:

$$W^{[L]}(\dots W^{[2]}(W^{[1]}\mathbf{x} + \mathbf{b}^{[1]}) \dots) = W'\mathbf{x} + \mathbf{b}'.$$

Las activaciones no lineales son lo que le dan **poder expresivo** a la red.

3.2 Las activaciones más comunes

Función	Fórmula	Rango	Uso típico
Sigmoid	$\sigma(z) = \frac{1}{1+e^{-z}}$	$(0, 1)$	Clasificación binaria (salida)
Tanh	$\tanh(z) = \frac{e^z - e^{-z}}{e^z + e^{-z}}$	$(-1, 1)$	Capas ocultas (antes de ReLU)
ReLU	$\max(0, z)$	$[0, \infty)$	Capas ocultas (estándar hoy)
Leaky ReLU	$\max(\alpha z, z), \alpha \ll 1$	$\mathbb{R}$	Evitar «neuronas muertas»
Softmax	$\frac{e^{z_i}}{\sum_j e^{z_j}}$	$(0, 1)$, suma 1	Clasificación multiclase (salida)

Intuición

¿Por qué ReLU domina?

- Sigmoid y Tanh *saturan*: cuando $|z|$ es grande, el gradiente se vuelve casi 0 $\rightarrow$ problema del **gradiente que desaparece** (*vanishing gradient*).
- ReLU no satura en la región positiva: su gradiente es exactamente 1. Esto hace que el entrenamiento de redes profundas sea mucho más estable.

4 Forward Propagation: El pase hacia adelante

Dado un ejemplo $\mathbf{x}$, el **forward pass** calcula la predicción recorriendo la red capa por capa:

$$\begin{aligned}\mathbf{a}^{[0]} &= \mathbf{x} \\ \mathbf{z}^{[\ell]} &= W^{[\ell]} \mathbf{a}^{[\ell-1]} + \mathbf{b}^{[\ell]}, \quad \ell = 1, \dots, L \\ \mathbf{a}^{[\ell]} &= f^{[\ell]}(\mathbf{z}^{[\ell]}) \\ \hat{\mathbf{y}} &= \mathbf{a}^{[L]}\end{aligned}$$

Ejemplo numérico — XOR con una MLP

Red: $[2 \rightarrow 2 \rightarrow 1]$ con activación sigmoid.

$$W^{[1]} = \begin{pmatrix} 20 & 20 \\ -20 & -20 \end{pmatrix}, \quad \mathbf{b}^{[1]} = \begin{pmatrix} -10 \\ 30 \end{pmatrix}, \quad W^{[2]} = (20 \ 20), \quad b^{[2]} = -30$$

Para $\mathbf{x} = (1, 1)^\top$:

$$\mathbf{z}^{[1]} = \begin{pmatrix} 20 \cdot 1 + 20 \cdot 1 - 10 \\ -20 \cdot 1 - 20 \cdot 1 + 30 \end{pmatrix} = \begin{pmatrix} 30 \\ -10 \end{pmatrix}$$

$$\mathbf{a}^{[1]} \approx \begin{pmatrix} \sigma(30) \\ \sigma(-10) \end{pmatrix} \approx \begin{pmatrix} 1.0 \\ 0.0 \end{pmatrix}$$

$$z^{[2]} = 20 \cdot 1 + 20 \cdot 0 - 30 = -10$$

$$\hat{y} = \sigma(-10) \approx 0.0 \quad \checkmark$$

XOR(1, 1) = 0. ¡La red lo resuelve!

5 Función de Costo Generalizada

Ya conocen el error cuadrático para un ejemplo:

$$\mathcal{L}_i = \frac{1}{2}(\hat{y}_i - y_i)^2.$$

Para un dataset de m ejemplos y una red con n_L salidas, la función de costo total es:

$$\mathcal{L} = \frac{1}{m} \sum_{i=1}^m \sum_{k=1}^{n_L} \frac{1}{2} \left(\hat{y}_k^{(i)} - y_k^{(i)} \right)^2$$

Para clasificación binaria se usa la **entropía cruzada** (cross-entropy), más apropiada con sigmoid:

$$\mathcal{L} = -\frac{1}{m} \sum_{i=1}^m \left[y^{(i)} \log \hat{y}^{(i)} + (1 - y^{(i)}) \log(1 - \hat{y}^{(i)}) \right]$$

6 Backpropagation: La regla de la cadena a escala

6.1 Idea central

El entrenamiento requiere $\frac{\partial \mathcal{L}}{\partial W^{[q]}}$ para cada capa. La función de costo es una composición:

$$\mathcal{L} = \mathcal{L}(\mathbf{a}^{[L]}(\mathbf{z}^{[L]}(W^{[L]}, \mathbf{a}^{[L-1]}(\dots))))$$

La **regla de la cadena** permite calcular estos gradientes eficientemente de atrás hacia adelante.

6.2 Derivación capa por capa

Definimos el **error de capa** $\delta^{[\ell]}$ como:

$$\delta^{[\ell]} = \frac{\partial \mathcal{L}}{\partial \mathbf{z}^{[\ell]}}$$

Gradientes de backprop

Capa de salida (con MSE):

$$\delta^{[L]} = \frac{\partial \mathcal{L}}{\partial \mathbf{a}^{[L]}} \odot f'^{[L]}(\mathbf{z}^{[L]}) = (\hat{\mathbf{y}} - \mathbf{y}) \odot f'^{[L]}(\mathbf{z}^{[L]})$$

Retropropagación a través de capa $\ell < L$:

$$\delta^{[\ell]} = (W^{[\ell+1]\top} \delta^{[\ell+1]}) \odot f'^{[\ell]}(\mathbf{z}^{[\ell]})$$

Gradientes de los parámetros:

$$\frac{\partial \mathcal{L}}{\partial W^{[\ell]}} = \delta^{[\ell]} \mathbf{a}^{[\ell-1]\top}, \quad \frac{\partial \mathcal{L}}{\partial \mathbf{b}^{[\ell]}} = \delta^{[\ell]}$$

donde $\odot$ es el producto elemento a elemento (Hadamard).

6.3 Algoritmo completo de entrenamiento

Algoritmo: Descenso del Gradiente con Backprop

Inicializar $W^{[\ell]}, \mathbf{b}^{[\ell]}$ aleatoriamente (pequeños)

Repetir por N_{epochs} épocas:

1. **Forward pass:** calcular $\mathbf{a}^{[1]}, \dots, \mathbf{a}^{[L]}$ y $\mathcal{L}$.
2. **Backward pass:** calcular $\delta^{[L]}, \dots, \delta^{[1]}$.
3. **Actualizar parámetros:**

$$W^{[\ell]} \leftarrow W^{[\ell]} - \eta \frac{\partial \mathcal{L}}{\partial W^{[\ell]}}, \quad \mathbf{b}^{[\ell]} \leftarrow \mathbf{b}^{[\ell]} - \eta \frac{\partial \mathcal{L}}{\partial \mathbf{b}^{[\ell]}}$$

7 Implementación: MLP desde cero en NumPy

A continuación se presenta una implementación modular y comentada. Analízala, entiende cada función y úsala como base para los ejercicios.

Listing 1: Clase MLP desde cero en NumPy

```
1 import numpy as np
```

```

2 import matplotlib.pyplot as plt
3
4 # Funciones de activacion
5
6 def relu(z):
7     return np.maximum(0, z)
8
9 def relu_grad(z):
10     return (z > 0).astype(float)
11
12 def sigmoid(z):
13     return 1.0 / (1.0 + np.exp(-np.clip(z, -500, 500)))
14
15 def sigmoid_grad(z):
16     s = sigmoid(z)
17     return s * (1 - s)
18
19 def tanh_grad(z):
20     return 1.0 - np.tanh(z)**2
21
22 ACTIVATIONS = {
23     'relu': (relu, relu_grad),
24     'sigmoid': (sigmoid, sigmoid_grad),
25     'tanh': (np.tanh, tanh_grad),
26     'linear': (lambda z: z, lambda z: np.ones_like(z)),
27 }
28
29 # Clase MLP
30
31 class MLP:
32     """
33     Red neuronal multicapa (MLP) implementada desde cero.
34
35     Parametros
36     -----
37     layer_sizes : list[int]
38         Tamano de cada capa, incluyendo entrada y salida.
39         Ej: [2, 4, 4, 1] -> 2 entradas, dos capas de 4, 1 salida.
40     activations : list[str]
41         Funcion de activacion para cada capa oculta + salida.
42         Ej: ['relu', 'relu', 'linear']
43     learning_rate : float
44         Tasa de aprendizaje eta.
45     """
46
47     def __init__(self, layer_sizes, activations, learning_rate=0.01):
48         assert len(activations) == len(layer_sizes) - 1
49         self.layer_sizes = layer_sizes
50         self.activations = activations
51         self.lr = learning_rate
52         self.L = len(layer_sizes) - 1 # numero de capas

```



```

50
51     # Inicializacion He (buena para ReLU)
52     self.W = []
53     self.b = []
54     for l in range(self.L):
55         fan_in = layer_sizes[l]
56         scale = np.sqrt(2.0 / fan_in)
57         self.W.append(np.random.randn(layer_sizes[l+1], layer_sizes
58                                     [l]) * scale)
59         self.b.append(np.zeros((layer_sizes[l+1], 1)))
60
61     # Forward pass
62
63     def forward(self, X):
64         """
65         X: (n_features, m) - columnas son ejemplos.
66         Devuelve y_hat y guarda activaciones para backprop.
67         """
68         self.Z = [] # pre-activaciones
69         self.A = [X] # activaciones (A[0] = X)
70
71         for l in range(self.L):
72             f, _ = ACTIVATIONS[self.activations[l]]
73             z = self.W[l] @ self.A[l] + self.b[l]
74             a = f(z)
75             self.Z.append(z)
76             self.A.append(a)
77
78         return self.A[-1] # y_hat
79
80     # Funcion de costo (MSE)
81
82     def cost(self, Y_hat, Y):
83         m = Y.shape[1]
84         return np.mean(0.5 * (Y_hat - Y)**2)
85
86     # Backward pass
87
88     def backward(self, Y):
89         m = Y.shape[1]
90         grads_W = [None] * self.L
91         grads_b = [None] * self.L
92
93         # Delta de la capa de salida
94         _, f_grad = ACTIVATIONS[self.activations[-1]]
95         delta = (self.A[-1] - Y) * f_grad(self.Z[-1])
96
97         for l in reversed(range(self.L)):
98             grads_W[l] = (delta @ self.A[l].T) / m

```

```

95         grads_b[l] = np.mean(delta, axis=1, keepdims=True)
96
97         if l > 0:
98             _, f_grad = ACTIVATIONS[self.activations[l-1]]
99             delta = (self.W[l].T @ delta) * f_grad(self.Z[l-1])
100
101         # Actualizar pesos
102         for l in range(self.L):
103             self.W[l] -= self.lr * grads_W[l]
104             self.b[l] -= self.lr * grads_b[l]
105
106         # Entrenamiento
107
108     def fit(self, X, Y, epochs=2000, verbose=True):
109         """
110         X: (features, m), Y: (outputs, m)
111         """
112         history = []
113         for epoch in range(epochs):
114             Y_hat = self.forward(X)
115             loss = self.cost(Y_hat, Y)
116             self.backward(Y)
117             history.append(loss)
118
119             if verbose and epoch % 500 == 0:
120                 print(f"Epoch {epoch:5d} | Loss: {loss:.6f}")
121
122         return history
123
124     def predict(self, X):
125         return self.forward(X)

```

Nota

Puntos clave de la implementación:

- Las matrices siguen la convención (**features, samples**): cada columna es un ejemplo.
- La inicialización *He* ($\sigma = \sqrt{2/n_{\text{in}}}$) es importante para ReLU; evita gradientes muy pequeños al inicio.
- `self.A[0] = X` permite que el bucle de backprop funcione uniformemente para todas las capas.

8 Ejercicios Propuestos: ¿Qué arquitectura necesita cada dataset?

Instrucciones generales

Para cada dataset a continuación:

1. Visualiza los datos con `matplotlib` antes de entrenar.
2. Diseña tu propia arquitectura (número de capas, neuronas por capa, funciones de activación).
3. Entrena con la clase MLP y grafica la curva de pérdida.
4. Visualiza la función aprendida sobre una malla 2D o en 1D.
5. **Reflexiona:** ¿qué pasó cuando usaste muy pocas capas? ¿Y cuando usaste demasiadas?

Compara con tus compañeros: dos personas con arquitecturas distintas pueden llegar a soluciones igualmente válidas.

Dataset 1 — Función Cuadrática

★

Objetivo: aprender $y = x^2$ en el intervalo $[-2, 2]$.

Generación del dataset:

```
1 np.random.seed(42)
2 X = np.random.uniform(-2, 2, (1, 500)) # 500 puntos
3 Y = X**2 + np.random.randn(1, 500)*0.05 # con ruido
4
5 # Normalizacion (buena practica)
6 X_norm = X / 2.0
7 Y_norm = Y / 4.0
```

Preguntas guía:

- ¿Cuántas capas necesitas para aproximar una parábola?
- ¿Funciona bien una red con una sola neurona oculta? ¿Con diez?
- ¿Qué activación tiene más sentido en la salida para una regresión?
- Grafica $\hat{y}$ vs y sobre datos de prueba. ¿Está bien calibrada la red?

Arquitectura de inicio sugerida (modifícala libremente): [1, 8, 1] con ReLU.

Dataset 2 — Campana Gaussiana

★★

Objetivo: aprender $y = e^{-x^2/2}$ (forma de campana).

Generación del dataset:

```
1 np.random.seed(0)
2 X = np.random.uniform(-4, 4, (1, 800))
3 Y = np.exp(-X**2 / 2.0) + np.random.randn(1, 800)*0.03
```

Preguntas guía:

- La función sube y baja: ¿puede ReLU simple capturar esto? ¿Qué sucede?
- Compara Tanh vs ReLU para capas ocultas. ¿Hay diferencia en velocidad de convergencia?
- ¿Cuántas épocas necesitas? Monitorea la curva de pérdida.
- Prueba con y sin normalización de X . ¿Cómo afecta al entrenamiento?

Pista: esta función es simétrica y acotada entre 0 y 1. Piensa qué función de activación en la salida respeta esa restricción.

Dataset 3 — Distribución sin Ecuación Cerrada

★★★

Objetivo: el reto es aprender una función irregular, construida como mezcla de señales.

Generación del dataset:

```
1 np.random.seed(7)
2 X = np.random.uniform(0, 10, (1, 1000))
3 Y = ( np.sin(X)
4       + 0.5 * np.sin(3*X)
5       + 0.3 * np.cos(5*X)
6       - 0.2 * (X/10)**2
7       + np.random.randn(1, 1000) * 0.1 )
8 # Normaliza X a [0,1] e Y a media 0
9 X_n = X / 10.0
10 Y_n = (Y - Y.mean()) / Y.std()
```

Preguntas guía:

- Esta función tiene variaciones a múltiples escalas. ¿Cómo afecta el número de neuronas?
- Divide los datos en entrenamiento (80%) y prueba (20%). ¿Hay sobreajuste (*overfitting*)?
- ¿Qué pasa si entrenas demasiado? ¿Y si entrenas muy poco?

- Grafica el error de entrenamiento y de prueba juntos. ¿En qué época divergen?

Meta extra: implementa un criterio de parada temprana (*early stopping*) cuando el error de prueba empiece a subir.

Dataset 4 — Clasificación Real: Cáncer de Mama (Wisconsin)

Objetivo: clasificar tumores como malignos o benignos a partir de características celulares.

Descripción: el dataset *Breast Cancer Wisconsin* contiene 569 muestras con 30 características extraídas de imágenes de biopsias (radio, textura, perímetro, área, etc.) y una etiqueta binaria (0 = maligno, 1 = benigno).

Carga del dataset:

```
1 from sklearn.datasets import load_breast_cancer
2 from sklearn.model_selection import train_test_split
3 from sklearn.preprocessing import StandardScaler
4
5 data = load_breast_cancer()
6 X_raw, y_raw = data.data, data.target # (569, 30), (569,)
7
8 # Normalizar
9 scaler = StandardScaler()
10 X_scaled = scaler.fit_transform(X_raw)
11
12 # Formato MLP: (features, samples)
13 X_tr, X_te, y_tr, y_te = train_test_split(
14     X_scaled, y_raw, test_size=0.2, random_state=42
15 )
16 X_tr = X_tr.T;    Y_tr = y_tr.reshape(1, -1)
17 X_te = X_te.T;    Y_te = y_te.reshape(1, -1)
```

Preguntas guía:

- Con 30 entradas y 1 salida binaria, ¿qué función de activación usas en la salida?
- Calcula la **exactitud** (*accuracy*) en el conjunto de prueba:

$$\text{accuracy} = \frac{\text{predicciones correctas}}{m_{\text{test}}}$$

- ¿Cuántas capas y neuronas necesitas para superar el 92% de exactitud?
- Construye la **matriz de confusión** manualmente con NumPy. ¿Cuántos falsos negativos (tumores malignos clasificados como benignos) tiene tu modelo?
- **Reflexión ética:** ¿importa más el recall de malignos o la precisión? ¿Por qué?

Código de evaluación:

```

1 # Predicción binaria con umbral 0.5
2 Y_pred = (model.predict(X_te) >= 0.5).astype(int)
3
4 # Exactitud
5 accuracy = np.mean(Y_pred == Y_te)
6 print(f"Accuracy: {accuracy*100:.2f}%")
7
8 # Matriz de confusion (manual)
9 TP = np.sum((Y_pred == 1) & (Y_te == 1))
10 TN = np.sum((Y_pred == 0) & (Y_te == 0))
11 FP = np.sum((Y_pred == 1) & (Y_te == 0))
12 FN = np.sum((Y_pred == 0) & (Y_te == 1))
13 print(f"TP={TP}  TN={TN}  FP={FP}  FN={FN}")

```

9 Resumen y Próximos Pasos

Concepto	Lo que aprendimos hoy
Límites del perceptrón	XOR no es linealmente separable; se necesitan capas
Arquitectura MLP	Capas de pesos $W^{[l]}$, sesgos $\mathbf{b}^{[l]}$ y activaciones $f^{[l]}$
Forward pass	Composición matricial de transformaciones no lineales
Funciones de activación	ReLU evita el vanishing gradient; sigmoid/tanh saturan
Backpropagation	Regla de la cadena propagada capa a capa (deltas $\delta^{[l]}$)
Implementación	MLP en NumPy puro: init He, forward, backward, update

Para la próxima sesión exploraremos:

- **Optimizadores avanzados:** Momentum, RMSProp, Adam — ¿por qué SGD puro queda corto?
- **Regularización:** Dropout y L2 — cómo combatir el sobreajuste.
- **Frameworks:** re-implementar todo en PyTorch y entender la diferenciación automática.

A Referencia rápida de derivadas

$f(z)$	$f'(z)$	Nota
$\sigma(z)$	$\sigma(z)(1 - \sigma(z))$	Satura para $ z > 5$
$\tanh(z)$	$1 - \tanh^2(z)$	Centrada en 0, mejor que sigmoid
$\text{ReLU}(z)$	$\mathbb{I}[z > 0]$	Gradiente exactamente 0 ó 1
z (lineal)	1	Para regresión en salida